# Vendor's Market — Complete Implementation Roadmap

**Project Vision:** Build a production-oriented commerce platform for Nepal that connects customers, retail vendors, and wholesale businesses through one secure system. The platform should offer a polished customer marketplace, a powerful vendor workspace, and a secure administrator console.

The implementation strategy is to build a reliable foundation first, deliver a useful marketplace early, and then expand into wholesale trading, advanced logistics, AI, and business intelligence without rewriting the application.

## 1. Core Product Roles

### Customer

- Discover nearby and distant shops and products.
- Compare prices, availability, delivery options, and vendor reputation.
- Save favorite products and shops.
- Chat with vendors and ask product questions.
- Place orders, pay using supported methods, and track deliveries.
- Submit reviews for eligible completed purchases.
- Request eligible returns, refunds, and dispute resolution.

### Vendor (seller, business buyer, and supplier)

- Register and manage one or more shops, subject to platform rules.
- Submit business information for verification.
- Create and manage product listings, images, prices, variants, and stock.
- Process retail orders and fulfill them.
- Publish wholesale offers and bulk pricing.
- Discover suppliers, request quotations, negotiate terms, and manage purchase orders.
- Coordinate stock transfers and bulk deliveries.
- Chat with customers and other vendors.
- Access inventory reports, sales analytics, and demand insights.
- Optionally invite shop staff with restricted permissions.
- **A vendor can both buy from and sell to other vendors using the same account. These are capabilities, not separate identities.**

### Platform Administrator

- Manage customer and vendor accounts and permissions.
- Review vendor verification applications.
- Moderate products, reviews, and reported content.
- Monitor orders, payments, commissions, refunds, and disputes.
- Manage categories, platform policies, promotions, and configuration.

- Review marketplace analytics and investigate suspicious activity.
- Maintain audit logs and secure administrative access.

## 2. Product Objectives and MVP Boundary

The first release should support a complete, trustworthy transaction:

- A vendor registers and creates a shop.
- The vendor publishes a product with accurate price and stock.
- A customer discovers the product and compares it with alternatives.
- The customer places an order using a supported payment or collection method.
- The vendor confirms and fulfills the order.
- The customer receives the product and can submit a review.
- The administrator can investigate issues and monitor the transaction.

## 3. Recommended Technology Stack

- **Frontend:** Next.js + TypeScript for the customer marketplace, vendor workspace, and admin console.
- **Backend:** FastAPI + Python for versioned REST APIs, business rules, permissions, order processing, wholesale trading, and integrations.
- **Database:** PostgreSQL + PostGIS for relational data and geographic discovery.
- **ORM and Migrations:** SQLAlchemy 2.x + Alembic.
- **Validation:** Pydantic.
- **UI Styling:** Tailwind CSS with a reusable accessible component system.
- **Authentication:** Secure session cookies or short-lived access tokens with properly managed refresh tokens.
- **Caching and Coordination:** Redis when justified by workload.
- **Images and Documents (Object Storage): MinIO High-Performance Object Storage (AGPLv3 / S3-Compatible).**

- **Public bucket:** Product photos, shop logos, and public media assets.

- **Private bucket:** Strictly private storage for vendor verification documents (PAN/VAT cards, business registration, citizenship certificates) accessed exclusively via FastAPI-generated short-lived pre-signed URLs.

- **Real-time Chat:** WebSockets with durable message storage.
- **Background Jobs:** Start with a simple worker; add Celery/Redis or another queue when needed.
- **Testing:** Pytest, HTTPX, Vitest, and Playwright.
- **Infrastructure:** Docker and CI/CD.
- **Monitoring:** Structured logs, error tracking, health checks, and performance metrics.
- **External Integrations:** Payment providers available to the business in Nepal, maps, delivery providers, email/SMS, and AI services through adapters.

**Architecture Recommendation:** Start with a modular monolith: one deployable backend service with clearly separated business modules. Avoid microservices until real scaling, team, or reliability needs justify their operational cost.

## 4. System Architecture

The system should follow this flow:

- **User Interfaces:** Customer marketplace, vendor workspace, admin console.

- **API Layer:** FastAPI handles validation, authentication, authorization, rate limiting, and business rules.
- **PostgreSQL:** Users, shops, products, stock, orders, transactions, messages, and audit data.
- **Object Storage (MinIO Cluster / Container):** Product photos (public) and encrypted business documents (private).
- **Messaging Infrastructure:** WebSockets and persisted messages.
- **Background Jobs:** Notifications, MinIO image processing/thumbnail generation, reports, and other asynchronous work.
- **External Adapters:** Payments, maps, delivery, email/SMS, and AI models.

The frontend must never connect directly to the production database or private MinIO buckets. Protected operations go through the backend, which verifies permissions and business rules.

# 5. Phased Implementation Roadmap

## Phase 0 — Product Definition and Design

- Finalize requirements, user stories, business rules, and MVP boundaries.
- Design navigation for customer, vendor, and administrator experiences.
- Create high-fidelity screens and reusable UI components.
- Define order states, cancellation rules, returns, and vendor verification.
- Document payment, delivery, wholesale, and review workflows.
- Define acceptance criteria for each user story.

***Completion Gate:*** *Each core workflow has an agreed design, clear acceptance criteria, and an owner.*

## Phase 1 — Project Foundation

- Initialize the Next.js frontend and FastAPI backend.
- Configure PostgreSQL, SQLAlchemy, Alembic, **MinIO**, and environment variables.
- Add **Docker Compose for local development (including `minio/minio` service)**.
- Implement structured logging, API error handling, configuration validation, and health checks.
- Set up linting, formatting, type checking, and automated test execution.
- Establish development, test, staging, and production configuration.
- Document setup in a README.

***Completion Gate:*** *A new developer can run the app locally using documented instructions; health checks, MinIO connection checks, and initial migrations pass.*

## Phase 2 — Identity and Access Control

- Registration, login, logout, session management, and password recovery.
- Email or phone verification using suitable providers.
- Customer profiles and multiple delivery addresses.
- Vendor registration and verification status (uploading registration files securely to **MinIO private buckets**).
- Role-based access control and resource-level authorization.
- Protected admin access and audit logging.
- Account suspension, deactivation, and privacy workflows.

***Completion Gate:*** *Users can access only their own resources and authorized shop operations, including through direct API requests.*

## Phase 3 — Product Catalogue and Discovery

- Categories, subcategories, brands, attributes, and product variants.
- Product images (uploaded to **MinIO public bucket**), descriptions, pricing, SKUs, and stock.
- Vendor storefronts and shop profiles.
- Search, sorting, filtering, and pagination.
- Location-based shop discovery.
- Product details, favorites, and wishlists.
- Listing moderation and availability validation.

***Completion Gate:*** *A vendor can publish a product, and customers can find it, view details, and compare it with other offers.*

## Phase 4 — Retail Ordering and Fulfillment

- Shopping cart and quantity management.
- Stock reservation and prevention of overselling.
- Checkout, address validation, and shipping calculations.
- Vendor-specific fulfillment orders for multi-vendor checkouts.
- Order confirmation, status transitions, cancellation rules, and returns.
- Payment integration or a clearly defined cash-on-delivery option where supported.
- Delivery tracking and customer notifications.
- Order history, invoices, and transaction records.

***Completion Gate:*** *A customer can place a valid order, a vendor can fulfill it, and the customer can track progress through completion.*

## Phase 5 — Communication and Reviews

- Customer-to-vendor conversations.
- Persisted chat history, unread counts, and notifications.
- Product and order references inside conversations.
- Verified-purchase reviews and ratings.
- Review reporting, moderation, and vendor responses.
- Support tickets and dispute escalation.

***Completion Gate:*** *Conversations remain available after reconnecting, and review eligibility is enforced by the backend.*

## Phase 6 — Wholesale and Vendor-to-Vendor Commerce

- Wholesale listings and quantity-based prices.
- Minimum order quantities and bulk availability.
- Supplier search and requests for quotations (RFQs).
- Quotations, negotiation history, and purchase orders.
- Trade-specific order states and delivery terms.
- Inter-vendor stock transfers and receipt confirmation.
- Supplier history, trade documents (stored in **MinIO**), and business reviews.

***Completion Gate:*** *A vendor can request a quote from another vendor, confirm a purchase order, receive the goods, and retain a complete transaction history.*

## Phase 7 — Administration and Operations

- User and vendor management.
- Verification queues and product moderation (viewing vendor documents via short-lived **MinIO pre-signed URLs**).
- Order, payment, and dispute oversight.
- Configurable categories, fees, policies, and promotional rules.
- Operational dashboards and reports.
- Administrative permission boundaries and audit trails.
- Fraud reporting and investigation workflows.

***Completion Gate:*** *Authorized staff can manage operations without bypassing security or making untraceable changes.*

## Phase 8 — Intelligence and Advanced Capabilities

- Personalized recommendations.
- Demand forecasting and inventory alerts.
- Visual product search and multilingual assistance.
- Price-drop alerts and group buying.
- Vendor performance insights and supplier matching.
- Advanced anomaly detection and marketplace intelligence.

***Completion Gate:*** *Features demonstrate measurable improvements against a baseline, respect data permissions, and have a fallback when AI results are unreliable.*

## Phase 9 — Launch and Continuous Improvement

- Staging deployment and production release.
- Security, accessibility, performance, and usability testing.
- Database and MinIO object store backup and restore drills.
- Error monitoring and operational alerts.
- Customer/vendor onboarding and support documentation.
- Pilot with a small group of real customers and vendors.
- Prioritized fixes based on observed problems and usage.

***Completion Gate:*** *The system can operate safely, recover from failures, and support real transactions with documented incident-response procedures.*

## 6. Project Structure (Updated with MinIO Storage Adapter)

```
vendors-market/
■■■ apps/
■   ■■■ web/
■   ■   ■■■ app/
■   ■   ■   ■■■ (public)/
■   ■   ■   ■■■ (customer)/
■   ■   ■   ■■■ vendor/
■   ■   ■   ■■■ admin/
■   ■   ■■■ components/
■   ■   ■   ■■■ ui/
■   ■   ■   ■■■ products/
■   ■   ■   ■■■ shops/
■   ■   ■   ■■■ checkout/
■   ■   ■   ■■■ dashboards/
■   ■   ■■■ lib/
■   ■   ■■■ hooks/
■   ■   ■■■ tests/
■   ■■■ backend/
■       ■■■ app/
■       ■   ■■■ main.py
■       ■   ■■■ core/
■       ■   ■■■ db/
■       ■   ■■■ common/
■       ■   ■■■ modules/
■       ■   ■   ■■■ auth/
■       ■   ■   ■■■ users/
■       ■   ■   ■■■ shops/
■       ■   ■   ■■■ catalog/
■       ■   ■   ■■■ inventory/
■       ■   ■   ■■■ carts/
■       ■   ■   ■■■ orders/
■       ■   ■   ■■■ payments/
■       ■   ■   ■■■ shipping/
■       ■   ■   ■■■ messaging/
■       ■   ■   ■■■ reviews/
■       ■   ■   ■■■ wholesale/
■       ■   ■   ■■■ admin/
■       ■   ■■■ integrations/
■       ■       ■■■ payments/
■       ■       ■■■ delivery/
■       ■       ■■■ storage/         <-- MinIO client & pre-signed URL generator
■       ■       ■■■ notifications/
■       ■■■ migrations/
■       ■■■ tests/
■■■ infra/
■   ■■■ docker/
■   ■   ■■■ minio/                   <-- MinIO setup & init scripts
■   ■■■ deployment/
■■■ docs/
■   ■■■ architecture.md
■   ■■■ api-conventions.md
■   ■■■ security.md
■   ■■■ runbook.md
■■■ docker-compose.yml
■■■ .env.example
■■■ README.md
```

## 7. Security and Reliability Updates for MinIO

- **MinIO Object Isolation:** Maintain two isolated buckets: `vendors-market-public` and `vendors-market-private`.
- **Pre-Signed URL Authorization:** Verification documents, invoices, and purchase order legal forms residing in `vendors-market-private` must never be directly downloadable via public URLs. FastAPI will issue short-lived pre-signed URLs (valid for 5–15 minutes) only after checking user or admin RBAC permissions.
- **MinIO Server-Side Encryption (SSE):** Enable SSE-S3 or SSE-KMS on MinIO instances to ensure encryption at rest for sensitive business registration documents.
- **Operational Backups:** Implement automated bucket mirroring (`mc mirror`) or snapshot routines to backup MinIO data alongside PostgreSQL dumps.